Brain Tumor MRI Classification Project

Project Overview

This project classifies brain MRI images into **four categories** using deep learning:

Class	Description
Glioma	A tumor arising from glial cells in the brain
Meningioma	A tumor that forms on the membranes covering the brain
No Tumor	Healthy brain with no detectable tumor
Pituitary	A tumor in the pituitary gland at the base of the brain

Five state-of-the-art deep learning models were trained and compared to find the best architecture for this task. A **Streamlit web application** was built to allow users to upload MRI scans and get real-time predictions from all five models simultaneously.

Project File Structure

```
Brain-MRI_classification/
├── app.py                          # Streamlit web application
├── requirements.txt                # Python dependencies
├── training_scripts/               # Jupyter notebooks for training (one per model)
│   ├── BrainComparision_resnet50.ipynb
│   ├── BrainComparision_densenet121.ipynb
│   ├── BrainComparision_EfficientNet-B0.ipynb
│   ├── BrainComparision_ConvNeXt-Tiny.ipynb
│   ├── BrainComparision_ViT-Small.ipynb
│   └── BrainMRI_checkpoints_testing.ipynb
├── models/                         # Saved model weights (.pth files)
│   ├── ResNet50_runA_best.pth
│   ├── DenseNet121_runB_best.pth
│   ├── EfficientNetB0_runC_best.pth
│   ├── ConvNeXtTiny_runD_best.pth
│   └── ViTSmall16_runE_best.pth
├── metrics/                        # Training history CSV files
├── test_results/                   # Confusion matrices, predictions, benchmarks
└── dataset/                        # MRI image data (Training/ and Testing/)
```

Dataset Used

The dataset consists of brain MRI images organized into four class folders:

- **Source**: Kaggle Brain Tumor MRI Dataset (Masoud Nickparvar)
- **Classes**: Glioma, Meningioma, No Tumor, Pituitary
- **Structure**: Pre-split into `Training/` and `Testing/` directories
- **Image format**: JPEG/PNG grayscale MRI scans
- **Total images**: Approximately 7,000+ MRI images

The dataset directory layout uses the **ImageFolder** convention:

```
dataset/
├── Training/
│   ├── glioma/
│   ├── meningioma/
│   ├── notumor/
│   └── pituitary/
└── Testing/
    ├── glioma/
    ├── meningioma/
    ├── notumor/
    └── pituitary/
```

Each subfolder name becomes the class label automatically when loaded with `torchvision.datasets.ImageFolder`.

Model Training Pipeline

All five models are trained using the **same pipeline** — only the model architecture name changes. The training runs on **Google Colab with GPU** (T4). Below is a detailed explanation of the entire training code.

Stage 1: Environment Setup and Imports

```
from google.colab import drive
drive.mount('/content/drive')
```

- **What it does**: Mounts Google Drive on Colab so we can access the dataset and save checkpoints.
- **Why needed**: The dataset and model weights are stored on Google Drive for persistence across sessions.
- **Pipeline stage**: Environment setup.

```
from pathlib import Path
DRIVE_ROOT = Path("/content/drive/MyDrive/BrainMRI_resnet50")
DRIVE_ROOT.mkdir(parents=True, exist_ok=True)
PROJECT_ROOT = DRIVE_ROOT
DATA_ROOT = str(DRIVE_ROOT / "brainMri")
CHECKPOINT_DIR = PROJECT_ROOT / "checkpoints"
METRICS_DIR    = PROJECT_ROOT / "metrics"
```

- **What it does**: Defines all project paths — where to find data and where to save models and metrics.
- **Why needed**: Keeps the project organized. `pathlib.Path` provides cross-platform path handling.
- **Pipeline stage**: Configuration.
- **Note**: The `DRIVE_ROOT` path changes for each model (e.g., `BrainMRI_resnet50`, `BrainMRI_densenet121`, etc.)

Stage 2: Library Imports

```
import torch
import torch.nn as nn
```

```python
import torch.nn.functional as F
import timm
from torchvision import transforms, datasets
from torch.utils.data import DataLoader
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.preprocessing import label_binarize
from tqdm.auto import tqdm
import pandas as pd
import numpy as np
import os, json
```

Import	Purpose
`torch`	Core PyTorch library for tensor operations and deep learning
`torch.nn`	Neural network layers and loss functions
`torch.nn.functional`	Functional API for operations like softmax
`timm`	**PyTorch Image Models** library — provides pretrained model architectures
`torchvision.transforms`	Image preprocessing and augmentation
`torchvision.datasets`	Dataset loaders (we use `ImageFolder`)
`DataLoader`	Batches data and feeds it to the model during training
`sklearn.metrics`	Calculates F1 score and AUC-ROC metrics
`label_binarize`	Converts labels to one-hot format for AUC calculation
`tqdm`	Progress bar for training loop visualization
`pandas`	Saves training history as CSV

Stage 3: System Configuration

```python
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
NUM_CLASSES   = 4
CLASS_NAMES   = ['glioma', 'meningioma', 'notumor', 'pituitary']
IMG_SIZE      = 224
BATCH_SIZE    = 16
NUM_EPOCHS    = 20
LR            = 1e-4
PATIENCE      = 5
```

Parameter	Value	Explanation
`device`	cuda/cpu	Automatically selects GPU if available, otherwise CPU
`NUM_CLASSES`	4	Four tumor classes to classify
`IMG_SIZE`	224	All images resized to 224×224 pixels (standard input for pretrained models)

BATCH_SIZE	16	Number of images processed together in one forward pass
NUM_EPOCHS	20	Maximum number of complete passes through the training dataset
LR	1e-4 (0.0001)	Learning rate — controls how much the model updates per step
PATIENCE	5	Early stopping patience — stops training if no improvement for 5 epochs

Stage 4: Data Preprocessing and Augmentation

```
train_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

val_transforms = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Line-by-line explanation:

1. `transforms.Compose([...])` — Chains multiple transforms into a single pipeline. Applied sequentially.
2. `transforms.Resize((224, 224))` — Resizes every image to 224×224 pixels.
   - *Why*: All pretrained models expect a fixed input size. 224×224 is the standard for ImageNet-pretrained models.
3. `transforms.RandomHorizontalFlip()` — Randomly flips the image horizontally with 50% probability.
   - *Why*: Data augmentation — brain tumors can appear on either side; this teaches the model not to depend on orientation.
4. `transforms.RandomRotation(15)` — Randomly rotates the image up to ±15 degrees.
   - *Why*: MRI scans may be slightly rotated; this makes the model more robust.
5. `transforms.ColorJitter(brightness=0.2, contrast=0.2)` — Randomly adjusts brightness and contrast.
   - *Why*: Simulates variation in MRI scan quality across different machines.
6. `transforms.ToTensor()` — Converts the PIL image to a PyTorch tensor and scales pixel values from [0, 255] to [0.0, 1.0].
   - *Why*: Neural networks operate on tensors (multi-dimensional arrays), not images.

7. `transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])` — Normalizes each RGB channel using ImageNet mean and standard deviation.
    - *Why*: The pretrained models were trained on ImageNet data with this normalization. Using the same normalization ensures the input distribution matches what the model expects.

Validation transforms skip augmentation (no flip, rotation, or jitter) because we want consistent evaluation results.

Stage 5: Dataset Loading

```
train_dataset = datasets.ImageFolder(root=DATA_ROOT + '/Training',
transform=train_transforms)
val_dataset   = datasets.ImageFolder(root=DATA_ROOT + '/Testing',  transform=val_transforms)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
num_workers=2, pin_memory=True)
val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False,
num_workers=2, pin_memory=True)
```

- `ImageFolder` : Automatically loads images from directory structure. Each subfolder name becomes a class label.
- `DataLoader` : Wraps the dataset to provide batched, shuffled, parallel-loaded data.
    - `shuffle=True` for training (randomizes order each epoch to prevent memorization).
    - `shuffle=False` for validation (consistent order for reproducible evaluation).
    - `num_workers=2` : Uses 2 background threads for parallel data loading.
    - `pin_memory=True` : Speeds up CPU-to-GPU data transfer.

Stage 6: Model Initialization (Transfer Learning)

```
model_name = "ResNet50_runA"
model = timm.create_model('resnet50', pretrained=True, num_classes=NUM_CLASSES)
model = model.to(device)
```

- `timm.create_model('resnet50', pretrained=True, num_classes=4)` :
    - Creates a ResNet50 architecture using the `timm` library.
    - `pretrained=True` loads weights trained on ImageNet (1.2 million images, 1000 classes).
    - `num_classes=4` automatically replaces the final classification layer (originally 1000 classes) with a new layer for 4 classes.
    - **This is Transfer Learning** — we reuse knowledge from ImageNet and fine-tune it for our task.
- `model.to(device)` : Moves the model to GPU for faster computation.

> ***The only line that changes between notebooks is the model name:***
>
> - *`'resnet50'` → ResNet50*
> - *`'densenet121'` → DenseNet121*
> - *`'efficientnet_b0'` → EfficientNet-B0*
> - *`'convnext_tiny'` → ConvNeXt-Tiny*
> - *`'vit_small_patch16_224'` → ViT-Small*

Stage 7: Loss Function and Optimizer

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
scaler = torch.cuda.amp.GradScaler()
```

- **`nn.CrossEntropyLoss()`** : The loss function for multi-class classification.
  - Combines `LogSoftmax` + `NLLLoss` internally.
  - Takes raw model outputs (logits) and true labels, returns a single loss value.
  - Penalizes wrong predictions more heavily.
- **`torch.optim.Adam(model.parameters(), lr=1e-4)`** : The Adam optimizer.
  - Updates model weights after each batch to minimize loss.
  - `model.parameters()` tells Adam which weights to update (all model weights).
  - `lr=1e-4` is the learning rate.
- **`GradScaler()`** : Enables mixed precision training (FP16) for faster computation on GPU.

Stage 8: Training Loop

```
best_val_loss = float('inf')
no_improve_epochs = 0

history = {
    "train_loss": [], "val_loss": [],
    "train_acc": [],  "val_acc": [],
    "train_f1": [],   "val_f1": [],
    "train_auc": [],  "val_auc": [],
}

for epoch in range(1, NUM_EPOCHS + 1):
    model.train()
    running_loss = 0.0
    all_preds, all_labels, all_probs = [], [], []

    pbar = tqdm(train_loader, desc=f"Epoch {epoch} [train]")
    for images, labels in pbar:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()

        with torch.cuda.amp.autocast():
            outputs = model(images)
            loss = criterion(outputs, labels)

        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()
```

```
        running_loss += loss.item() * images.size(0)
        probs = F.softmax(outputs, dim=1)
        preds = probs.argmax(dim=1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
        all_probs.extend(probs.detach().cpu().numpy())
```

Line-by-line explanation:

1. `best_val_loss = float('inf')` — Initialize best validation loss to infinity. Any real loss will be better.
2. `no_improve_epochs = 0` — Counter for early stopping. Counts epochs without improvement.
3. `history = {...}` — Dictionary to store metrics per epoch for later analysis.
4. `for epoch in range(1, NUM_EPOCHS + 1)` — Loop through 20 epochs.
5. `model.train()` — Sets the model to training mode (enables dropout and batch normalization updates).
6. `running_loss = 0.0` — Accumulates total loss across all batches in one epoch.
7. `for images, labels in pbar` — Iterates over batches from the DataLoader.
8. `images, labels = images.to(device), labels.to(device)` — Moves batch data to GPU.
9. `optimizer.zero_grad()` — Clears old gradients. Required before each backward pass; otherwise gradients accumulate.
10. `with torch.cuda.amp.autocast()` — Enables automatic mixed precision (FP16) for faster forward pass.
11. `outputs = model(images)` — Forward pass: feeds images through the model, gets raw predictions (logits).
12. `loss = criterion(outputs, labels)` — Computes cross-entropy loss between predictions and true labels.
13. `scaler.scale(loss).backward()` — Backward pass: computes gradients of the loss with respect to all model weights.
14. `scaler.step(optimizer)` — Updates model weights using the computed gradients.
15. `scaler.update()` — Adjusts the gradient scaler for mixed precision.
16. `probs = F.softmax(outputs, dim=1)` — Converts raw logits to probabilities (0 to 1, summing to 1).
17. `preds = probs.argmax(dim=1)` — Gets the predicted class (index with highest probability).

Stage 9: Validation Loop

```
    model.eval()
    val_loss = 0.0
    val_preds, val_labels, val_probs = [], [], []

    with torch.no_grad():
        pbar_val = tqdm(val_loader, desc=f"Epoch {epoch} [eval]")
```

```
    for images, labels in pbar_val:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        val_loss += loss.item() * images.size(0)

        probs = F.softmax(outputs, dim=1)
        preds = probs.argmax(dim=1)

        val_preds.extend(preds.cpu().numpy())
        val_labels.extend(labels.cpu().numpy())
        val_probs.extend(probs.cpu().numpy())
```

- `model.eval()` — Sets model to evaluation mode (disables dropout, freezes batch normalization).
- `torch.no_grad()` — Disables gradient computation. Saves memory and speeds up inference since we do not need to update weights during validation.
- The rest follows the same pattern as training but **without** `optimizer.zero_grad()`, `backward()`, or `step()`.

Stage 10: Metrics Calculation

```
# Calculate metrics
t_acc = np.mean(np.array(all_preds) == np.array(all_labels))
v_acc = np.mean(np.array(val_preds) == np.array(val_labels))
t_f1  = f1_score(all_labels, all_preds, average='macro')
v_f1  = f1_score(val_labels, val_preds, average='macro')

y_true_bin = label_binarize(val_labels, classes=list(range(NUM_CLASSES)))
y_prob     = np.array(val_probs)
v_auc = roc_auc_score(y_true_bin, y_prob, average='macro', multi_class='ovr')
```

- **Accuracy**: Proportion of correct predictions.
- **F1 Score** (macro): Harmonic mean of precision and recall, averaged across all classes. Better metric when classes are imbalanced.
- **AUC-ROC**: Area Under the Receiver Operating Characteristic curve. Measures how well the model distinguishes between classes. 1.0 = perfect.
- `label_binarize`: Converts class labels to one-hot encoding for multi-class AUC calculation.

Stage 11: Early Stopping and Model Saving

```
# Save best model
if val_loss < best_val_loss:
    best_val_loss = val_loss
    no_improve_epochs = 0
    torch.save(model.state_dict(), CHECKPOINT_DIR / f"{model_name}_best.pth")
    print("✅ Saved best model!")
else:
    no_improve_epochs += 1
    if no_improve_epochs >= PATIENCE:
```

```
            print("■ Early stopping triggered!")
            break
```

- **Best model saving**: If validation loss improves, save the model weights as a `.pth` file. This ensures we keep the best-performing version.
- `model.state_dict()` : Saves only the model parameters (not the architecture). This is the standard PyTorch way.
- **Early stopping**: If validation loss does not improve for 5 consecutive epochs (`PATIENCE = 5`), training stops early to prevent overfitting.

Stage 12: Saving Training History

```
history_df = pd.DataFrame(history)
history_df.to_csv(METRICS_DIR / f"{model_name}_history.csv", index=False)
```

- Saves the per-epoch metrics (loss, accuracy, F1, AUC) as a CSV file.
- Used later for plotting training curves in the Streamlit app.

Deep Learning Models Used

All five models are available in the `timm` (PyTorch Image Models) library and are loaded with ImageNet pretrained weights.

1. ResNet50 (Residual Network — 50 layers)

Attribute	Value
Architecture	Deep CNN with residual (skip) connections
Parameters	~23.52 million
GFLOPs	4.13
Test Accuracy	99.47%
F1 Score	0.9946
Key Innovation	Skip connections solve the vanishing gradient problem

Why chosen: ResNet50 is a proven, reliable architecture. The skip connections allow training very deep networks by letting gradients flow directly through shortcut paths. It is the baseline standard for image classification benchmarks.

Architecture overview: Input → Conv layers → 4 residual blocks (each with bottleneck layers: 1×1, 3×3, 1×1 convolutions + skip connection) → Global Average Pooling → Fully Connected Layer (4 classes).

---

2. DenseNet121 (Densely Connected Network — 121 layers)

Attribute	Value
Architecture	CNN with dense connections (every layer connects to every other)
Parameters	~6.96 million

GFLOPs	2.83
Test Accuracy	99.39%
F1 Score	0.9934
Key Innovation	Feature reuse through dense connections

Why chosen: DenseNet has significantly fewer parameters than ResNet50 but achieves comparable accuracy. Each layer receives feature maps from all preceding layers, promoting feature reuse and reducing redundancy.

Advantage: Very parameter-efficient. Uses only ~7M parameters vs ResNet's ~24M, making it faster to train and less prone to overfitting on smaller datasets.

---

3. EfficientNet-B0 (Efficient Network — Baseline)

Attribute	Value
Architecture	CNN designed by Neural Architecture Search (NAS)
Parameters	~4.01 million
GFLOPs	0.38
Test Accuracy	99.62%
F1 Score	0.9959
Key Innovation	Compound scaling of depth, width, and resolution

Why chosen: EfficientNet-B0 achieves the **second-best accuracy** with the **fewest parameters** (only 4M) and lowest computational cost (0.38 GFLOPs). It uses compound scaling to balance network depth, width, and input resolution optimally.

Advantage: Best efficiency-to-accuracy ratio. Ideal for deployment on resource-constrained devices.

---

4. ConvNeXt-Tiny (Modernized ConvNet)

Attribute	Value
Architecture	Pure CNN modernized with Transformer-era design choices
Parameters	~27.82 million
GFLOPs	4.45
Test Accuracy	**99.77%** (best)
F1 Score	**0.9975** (best)
Key Innovation	CNN designs inspired by Vision Transformers

Why chosen: ConvNeXt-Tiny achieves the **highest accuracy** (99.77%) among all five models. It is a pure convolutional network that incorporates design principles from Vision Transformers (layer normalization, larger kernel sizes, depthwise convolutions).

Advantage: Best performing model overall. Combines the efficiency of CNNs with modern architectural innovations.

---

5. ViT-Small (Vision Transformer — Small)

Attribute	Value
Architecture	Transformer applied to image patches
Parameters	~21.67 million
GFLOPs	4.24
Test Accuracy	98.25%
F1 Score	0.9815
Key Innovation	Self-attention mechanism on image patches

Why chosen: ViT brings the Transformer architecture (originally designed for NLP) to computer vision. It splits the image into 16×16 patches, treats each patch as a "token", and applies self-attention to learn global relationships between patches.

Advantage: Can capture long-range dependencies in images that CNNs might miss. However, Transformers generally need more data to outperform CNNs.

How the Same Code Works for All Five Models

The only change between the five training notebooks is **one line**:

```
# ResNet50 notebook:
model = timm.create_model('resnet50', pretrained=True, num_classes=4)

# DenseNet121 notebook:
model = timm.create_model('densenet121', pretrained=True, num_classes=4)

# EfficientNet-B0 notebook:
model = timm.create_model('efficientnet_b0', pretrained=True, num_classes=4)

# ConvNeXt-Tiny notebook:
model = timm.create_model('convnext_tiny', pretrained=True, num_classes=4)

# ViT-Small notebook:
model = timm.create_model('vit_small_patch16_224', pretrained=True, num_classes=4)
```

The `timm.create_model()` function handles everything — downloading pretrained weights, building the architecture, and replacing the classifier head for 4 classes. This is why the same training pipeline works seamlessly across all models.

Model Comparison Summary

Model	Parameters	Accuracy	F1 Score	AUC	Inference Time
ConvNeXt-Tiny	27.82M	**99.77%**	**0.9975**	**1.0000**	1.72ms

EfficientNet-B0	4.01M	99.62%	0.9959	1.0000	3.15ms
ResNet50	23.52M	99.47%	0.9946	0.9999	2.20ms
DenseNet121	6.96M	99.39%	0.9934	0.9999	3.87ms
ViT-Small	21.67M	98.25%	0.9815	0.9993	1.67ms

Training Decisions

Why Adam Optimizer?

Adam (Adaptive Moment Estimation) was chosen because:

1. **Adaptive learning rates**: Adam maintains a separate learning rate for each parameter, adjusting automatically based on gradient history. No manual tuning needed.
2. **Momentum + RMSProp combined**: Adam combines the benefits of two other optimizers — Momentum (smooths gradients) and RMSProp (adapts per-parameter learning rates).
3. **Works well with transfer learning**: When fine-tuning pretrained models, Adam handles the different learning requirements of frozen vs. unfrozen layers effectively.
4. **Fast convergence**: Adam typically converges faster than SGD on complex tasks.

How it works (simple terms): After each batch, Adam calculates the gradient (direction to adjust weights). It keeps a running average of past gradients (momentum) and past squared gradients (variance). It uses these to compute a smart update — large updates for rarely-updated parameters, small updates for frequently-updated ones.

Why Learning Rate = 1e-4 (0.0001)?

- **Too high** (e.g., 0.01): The model would overshoot the optimal weights and may never converge.
- **Too low** (e.g., 0.00001): Training would be extremely slow.
- **1e-4 is the sweet spot** for fine-tuning pretrained models. The pretrained weights are already close to a good solution, so we only need small adjustments.

Why CrossEntropyLoss?

- **Standard for multi-class classification**: When you have one correct class out of multiple options, CrossEntropyLoss is the go-to choice.
- It combines `log_softmax` (converts logits to log-probabilities) and `negative log likelihood` into one efficient operation.
- **Penalizes confident wrong predictions** more heavily than uncertain ones.

Why Batch Size = 16?

- **GPU memory constraint**: Larger batch sizes require more GPU memory. On a T4 GPU (15GB), batch size 16 balances memory usage and training speed.
- Smaller batches introduce beneficial noise (regularization effect).
- Standard choice for medical imaging tasks.

Why 20 Epochs with Early Stopping (Patience=5)?

- **20 epochs** provides enough iterations for the model to converge.
- **Early stopping** prevents overfitting: if the validation loss does not improve for 5 consecutive epochs, training stops automatically.
- In practice, most models converged well before 20 epochs.

Why Transfer Learning?

- **Medical datasets are typically small** (~7,000 images). Training a deep network from scratch on this data would lead to overfitting.
- **Pretrained ImageNet weights** provide the model with general visual features (edges, textures, shapes) learned from 1.2 million images.
- We only need to **fine-tune** these features for our specific task (brain tumor detection).
- Transfer learning dramatically reduces training time (minutes instead of hours/days).

Why Pretrained Weights?

- Models pretrained on ImageNet have already learned hierarchical features:
  - **Early layers**: Detect edges, corners, basic textures
  - **Middle layers**: Detect more complex patterns (circles, grids)
  - **Deep layers**: Detect high-level features (object parts)
- These generic features transfer well to medical imaging tasks.
- Only the final classification layer needs to be trained from scratch for our 4 classes.

Streamlit Application

The file `app.py` is a **Streamlit web dashboard** with three pages. Below is a line-by-line explanation.

Import Statements (Lines 2–10)

```
import streamlit as st          # Web framework for building the dashboard
import torch                    # PyTorch for model inference
import timm                     # To recreate model architectures
from PIL import Image           # To open and process uploaded images
from torchvision import transforms  # Image preprocessing pipeline
import os                       # File system operations
import pandas as pd             # DataFrames for tabular data
import plotly.express as px     # Interactive charts
import plotly.graph_objects as go  # Advanced chart customization
```

Page Configuration (Lines 13–18)

```
st.set_page_config(
    page_title="NeuroScan Dashboard",
    page_icon="🧠",
    layout="wide",
    initial_sidebar_state="expanded"
)
```

Sets the browser tab title, icon, and uses wide layout for better visualization.

Constants (Lines 21–53)

```
CLASS_NAMES = ['Glioma', 'Meningioma', 'No Tumor', 'Pituitary']
```

```
MODELS_CONFIG = {
    "ConvNeXt-Tiny": {
        "id": "convnext_tiny",
        "path": "models/ConvNeXtTiny_runD_best.pth",
        "history": "metrics/ConvNeXtTiny_runD_history.csv",
        "cm_path": "test_results/ConvNeXtTiny_runD_cm.png"
    },
    # ... (same structure for all 5 models)
}
```

- `CLASS_NAMES` : Maps model output indices (0, 1, 2, 3) to human-readable tumor types.
- `MODELS_CONFIG` : Dictionary containing each model's `timm` model ID, saved weights path, training history CSV, and confusion matrix image path.

Model Loading Function (Lines 76–96)

```
@st.cache_resource
def load_models():
    loaded_models = {}
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    for display_name, config in MODELS_CONFIG.items():
        model_path = config["path"]
        if not os.path.exists(model_path):
            continue
        model = timm.create_model(config["id"], pretrained=False,
num_classes=len(CLASS_NAMES))
        state_dict = torch.load(model_path, map_location=device)
        model.load_state_dict(state_dict)
        model.to(device)
        model.eval()
        loaded_models[display_name] = model

    return loaded_models, device
```

- **`@st.cache_resource`** : Caches loaded models so they are not reloaded on every page interaction. Critical for performance.
- **`pretrained=False`** : We do NOT download ImageNet weights here. Instead, we load our own trained weights from `.pth` files.
- **`torch.load(model_path, map_location=device)`** : Loads the saved weight dictionary.
- **`model.load_state_dict(state_dict)`** : Applies the saved weights to the model architecture.
- **`model.eval()`** : Sets the model to evaluation mode (disables dropout, freezes batch norm).

Image Preprocessing Function (Lines 98–104)

```
def preprocess_image(image):
    transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

```
    ])
    return transform(image).unsqueeze(0)
```

- **Same normalization** as training (ImageNet mean/std) — this is critical for correct predictions.
- `unsqueeze(0)` : Adds a batch dimension. Model expects input shape `[batch, channels, height, width]` , so a single image `[3, 224, 224]` becomes `[1, 3, 224, 224]` .

Inference Page (Lines 113–169)

```
uploaded_file = st.file_uploader("Upload MRI Image", type=["jpg", "jpeg", "png"])

if uploaded_file:
    image = Image.open(uploaded_file).convert('RGB')
    input_tensor = preprocess_image(image).to(device)

    results = []
    with torch.no_grad():
        for name, model in models.items():
            outputs = model(input_tensor)
            probs = torch.nn.functional.softmax(outputs, dim=1)
            conf, pred_cls = torch.max(probs, 1)
            results.append({
                "Model": name,
                "Prediction": CLASS_NAMES[pred_cls.item()],
                "Confidence": conf.item()
            })
```

- `st.file_uploader` : Creates an upload button for MRI images.
- `Image.open().convert('RGB')` : Opens the uploaded image and converts to RGB (3 channels). MRI scans may be grayscale; this converts them to 3-channel format.
- `torch.no_grad()` : Disables gradient computation during inference (saves memory).
- **Inference loop**: Runs the preprocessed image through all 5 models sequentially.
- `softmax(outputs, dim=1)` : Converts raw logits to probabilities.
- `torch.max(probs, 1)` : Gets the highest probability (confidence) and its index (predicted class).

Models Overview Page (Lines 173–211)

Displays benchmark data in a leaderboard table with interactive charts:

- Accuracy comparison bar chart
- Efficiency scatter plot (parameters vs accuracy, bubble size = GFLOPs)
- Inference speed horizontal bar chart

Detailed Analysis Page (Lines 215–274)

Allows selecting a specific model to view:

- Training curves (loss and accuracy over epochs) with dual Y-axes
- Final epoch metrics (Val Accuracy, Val AUC, Val F1, Val Loss)
- Confusion matrix image

Prediction Workflow

Here is the complete step-by-step flow from user upload to final classification:

```
Step 1: User uploads an MRI image (JPG/PNG) via the Streamlit file uploader.
         ↓
Step 2: Image is opened with PIL and converted to RGB (3 channels).
         ↓
Step 3: Image is preprocessed:
         • Resized to 224 × 224 pixels
         • Converted to a PyTorch tensor (pixel values scaled to 0–1)
         • Normalized with ImageNet mean and standard deviation
         • Batch dimension added (shape: [1, 3, 224, 224])
         ↓
Step 4: Preprocessed tensor is moved to GPU (if available).
         ↓
Step 5: The tensor is passed through each of the 5 loaded models
         (forward pass in evaluation mode with no gradient tracking).
         ↓
Step 6: Each model outputs raw logits (4 values, one per class).
         ↓
Step 7: Softmax converts logits to probabilities (4 values summing to 1.0).
         ↓
Step 8: The class with the highest probability is selected as the prediction.
         The probability value becomes the confidence score.
         ↓
Step 9: Results are displayed as metric cards and a confidence bar chart.
         Each model shows: Prediction (e.g., "Glioma") + Confidence (e.g., 98.7%)
```

Important Concepts

CNN (Convolutional Neural Network)

A type of neural network specialized for image processing. Uses **convolutional layers** that slide small filters (kernels) across the image to detect features like edges, textures, and patterns. Deeper layers detect increasingly complex features. CNNs learn these filters automatically during training.

Transfer Learning

The practice of taking a model trained on a large dataset (e.g., ImageNet — 1.2M images) and reusing its learned features for a different but related task (e.g., brain tumor classification). Instead of training from scratch, we start with pretrained weights and fine-tune the model on our specific data. This works because low-level features (edges, textures) are universal across visual tasks.

Optimizer

An algorithm that updates the model's weights during training to minimize the loss function. **Adam** (used here) is an adaptive optimizer that automatically adjusts learning rates per parameter. It combines momentum (smoothing) and adaptive learning rates for efficient training.

Loss Function

A mathematical function that measures how wrong the model's predictions are. **CrossEntropyLoss** (used here) is standard for multi-class classification. It outputs a high value when the model is confidently wrong and a low value when the model is correct. The goal of training is to minimize this value.

Image Preprocessing

The steps applied to raw images before feeding them to a neural network:

1. **Resize**: Make all images the same fixed size (224×224)
2. **ToTensor**: Convert pixels from 0–255 integers to 0.0–1.0 floats
3. **Normalize**: Adjust pixel values to match the distribution the pretrained model was trained on (ImageNet statistics)

Data Augmentation

Artificially increasing the training dataset by applying random transformations (flips, rotations, color changes) to existing images. This teaches the model to be robust to variations and prevents overfitting (memorizing training data).

Softmax

A function that converts raw model outputs (logits) into probabilities. It takes a vector of arbitrary values and transforms them into positive values that sum to 1.0. The class with the highest probability is the model's prediction.

Possible Viva Questions

Q1: What is the purpose of this project?

A: To classify brain MRI images into four categories (Glioma, Meningioma, No Tumor, Pituitary) using deep learning, and compare the performance of five different model architectures.

Q2: Why did you use transfer learning instead of training from scratch?

A: Our medical dataset has only ~7,000 images. Training a deep network (23M+ parameters) from scratch on this data would lead to severe overfitting. Transfer learning uses pretrained ImageNet weights that already know general visual features like edges, textures, and shapes. We only need to fine-tune these features for brain tumor detection, which is much more data-efficient.

Q3: What is the `timm` library and why was it used?

A: `timm` (PyTorch Image Models) is a library that provides hundreds of pretrained model architectures with a unified API. It allowed us to switch between 5 different architectures (ResNet50, DenseNet121, EfficientNet-B0, ConvNeXt-Tiny, ViT-Small) by changing just one line of code — the model name string. This made our comparison fair and consistent.

Q4: Which model performed the best and why?

A: **ConvNeXt-Tiny** achieved the best accuracy (99.77%) and F1 score (0.9975). It is a modern pure CNN that incorporates design principles from Vision Transformers (larger kernels, layer normalization, depthwise convolutions), combining the best of both architectures.

Q5: What is early stopping and why was it used?

A: Early stopping monitors the validation loss during training. If the loss does not improve for a set number of epochs (patience=5 in our case), training stops automatically. This prevents overfitting — the model won't continue training and start memorizing the training data instead of learning general patterns.

Q6: Why did you normalize images with `mean=[0.485, 0.456, 0.406]` and `std=[0.229, 0.224, 0.225]`?

A: These are the mean and standard deviation of the ImageNet dataset (which has 1.2 million images). Since our pretrained models were trained on ImageNet, we must use the same normalization to ensure the input data distribution matches what the model expects. Using different statistics would degrade performance.

Q7: What loss function was used and why?

A: CrossEntropyLoss — the standard loss function for multi-class classification. It combines LogSoftmax and Negative Log Likelihood. It penalizes the model more when it makes confident wrong predictions, effectively encouraging the model to be accurate and well-calibrated.

Q8: What is the difference between `model.train()` and `model.eval()`?

A: `model.train()` enables training-specific behaviors like dropout (randomly disabling neurons to prevent overfitting) and batch normalization (updating running statistics). `model.eval()` disables these — dropout is turned off and batch norm uses the learned running statistics. Always use `model.eval()` during inference and validation.

Q9: Why is `torch.no_grad()` used during inference?

A: `torch.no_grad()` disables gradient calculation. During inference, we don't need gradients (since we're not training/updating weights). Disabling them saves GPU memory and speeds up computation.

Q10: What does the `unsqueeze(0)` do in the preprocessing?

A: The model expects a batch of images with shape `[batch_size, channels, height, width]`. A single image has shape `[3, 224, 224]`. `unsqueeze(0)` adds a batch dimension at position 0, making it `[1, 3, 224, 224]` — a batch of one image.

Q11: How does the Streamlit app load the trained models?

A: The app uses `timm.create_model()` with `pretrained=False` to create the same architecture, then loads the trained weights from `.pth` files using `torch.load()` and `model.load_state_dict()`. The `@st.cache_resource` decorator ensures models are loaded only once and cached for all subsequent requests.

Q12: What is the difference between ResNet and DenseNet architectures?

A: **ResNet** uses **skip connections** (adds the input of a block to its output), allowing gradients to flow through shortcut paths. **DenseNet** uses **dense connections** where every layer receives feature maps from *all* preceding layers (concatenation instead of addition). DenseNet achieves similar accuracy with far fewer parameters (7M vs 24M) due to feature reuse.

Q13: Why is EfficientNet-B0 considered efficient?

A: EfficientNet uses **compound scaling** — it jointly scales the network's depth, width, and resolution using a set of fixed coefficients found by Neural Architecture Search. This balanced scaling achieves higher accuracy per FLOP compared to models that scale only one dimension. B0 has only 4M parameters but achieves 99.62% accuracy.

Q14: What is a Vision Transformer (ViT) and how is it different from CNN?

A: A ViT splits the image into fixed-size patches (16×16), treats each patch as a "token" (like words in NLP), and processes them using the Transformer's **self-attention** mechanism. Unlike CNNs that use local convolutional filters, ViT can capture global relationships between distant image regions. However, ViTs typically need more data to outperform CNNs — which is why ViT-Small had the lowest accuracy (98.25%) in our experiment.

Q15: What would you do to improve the model further?

A: Several approaches could improve results:

1. **Larger dataset**: More training images would especially help ViT.
2. **Learning rate schedulers**: Use cosine annealing or step decay to reduce learning rate during training.
3. **Ensemble methods**: Combine predictions from multiple models (majority voting or averaging probabilities).
4. **Test-time augmentation**: Apply transforms during inference and average predictions.
5. **Fine-tune in stages**: First train only the classifier head, then gradually unfreeze and fine-tune deeper layers.
6. **Cross-validation**: Use k-fold cross-validation instead of a single train/test split for more robust evaluation.

Document generated for project review preparation.

Literature Survey

This section provides an overview of ten recent research papers in the domain of Brain MRI classification. Understanding these papers provides a broader context for the architectural choices and methodologies used in our project.

1. Enhancing brain tumor classification through ensemble attention mechanism

Authors: Fatih Celik, Kemal Celik, & Ayse Celik **Published:** 2024 (Scientific Reports) **Abstract/Brief:** This paper introduces an ensemble attention mechanism that extracts intermediate- and final-level feature maps from MobileNetV3 and EfficientNetB7. It uses a co-attention mechanism to direct focus onto critical regions to extract global features, improving the classification of diverse anatomical structures in MRI images. Tested on the BraTS 2019 and Figshare datasets, it achieved high accuracy (up to 98.94%), effectively isolating tumor regions from normal brain tissue.

2. Lightweight transfer learning models for multi-class brain tumor classification

Authors: A. Gorenshtein et al. **Published:** 2025 (Journal of Imaging Informatics in Medicine) **Abstract/Brief:** The authors explored lightweight deep learning models (ResNet-18, 34, 50, and a custom CNN) to classify glioma, meningioma, pituitary tumors, and healthy MRIs. By utilizing pre-trained networks and a dataset of 7023 images, their ResNet architectures achieved nearly perfect validation metrics (98.5–99.2% accuracy) while maintaining computational efficiency suitable for real-time diagnostic environments.

3. EFF_D_SVM: a robust multi-type brain tumor classification system

Authors: Jincan Zhang et al. **Published:** 2023 (Frontiers in Neuroscience) **Abstract/Brief:** This research proposes a hybrid classification system named EFF_D_SVM. It utilizes a pre-trained EfficientNetB0 model as a feature extractor, replacing the classification layer with custom dropout and dense layers. These extracted features are then classified using a Support Vector Machine (SVM). Verified with Grad-CAM for interpretability, the hybrid approach improves robustness in handling multi-type brain tumors.

4. Pre-trained deep learning models for brain MRI image classification

Authors: S. Krishnapriya and Y. Karuna **Published:** 2023 (Frontiers in Human Neuroscience) **Abstract/Brief:** This study evaluates several established pre-trained models﹐including VGG16, VGG19, ResNet50, and InceptionV3﹐for classifying brain MRI images. It emphasizes the advantage of avoiding manual feature extraction and the power of transfer learning in medical imaging, though it also notes the risk of overfitting when these complex models are applied to relatively small datasets.

5. Advanced brain tumor classification in MR images using transfer learning

Authors: R. Disci et al. (Assuming corresponding paper content from survey) **Published:** 2024/2025 **Abstract/Brief:** This paper investigates the effectiveness of several pre-trained models (Xception, MobileNetV2, InceptionV3, ResNet50, VGG16, and DenseNet121) on a public dataset of 7023 MRI images. Xception emerged as the top performer (98.73% accuracy), demonstrating exceptional generalization capabilities and effectiveness in addressing class imbalances, though the authors highlight the ongoing challenge of model interpretability.

6. Hybrid Model using Attention Mechanisms for Brain Tumor Categorization

Authors: Literature Survey ID [6] **Published:** 2024 **Abstract/Brief:** This work focuses on combining MobileNetV3Large, InceptionV3, and EfficientNetB0 architectures into a hybrid model. The inclusion of a channel-wise attention mechanism enhances the discriminative power, achieving a 97.94% accuracy. The study highlights the revolutionary potential of combining hybrid architectures with attention processes for MRI classification.

7. Brain Tumor Classification via ML and Transfer Learning

Authors: Literature Survey ID [7] (S. M. Malakouti et al.) **Published:** 2024 **Abstract/Brief:** This research applied both traditional machine learning (LightGBM, Random Forest, SVM) on numerical data and transfer learning (modified GoogLeNet) on MRI images. While LightGBM achieved 95.7% accuracy on numerical data, the modified GoogLeNet boosted image classification accuracy to 99.3%, demonstrating the synergy of ML and deep learning.

8. Review on deep learning methods for brain tumor segmentation and classification

Authors: H. Tandel et al. **Published:** 2020 (Computers in Biology and Medicine) **Abstract/Brief:** A comprehensive review paper that catalogs various CNN-based methods developed for brain tumor classification. It surveys works tested on BRATS, Figshare, and Kaggle datasets, providing a strong theoretical overview of the state-of-the-art without introducing a novel experimental model.

9. Integrated approach of federated learning with transfer learning

Authors: E. Albalawi et al. **Published:** 2024 (BMC Medical Imaging) **Abstract/Brief:** To address the critical issue of patient privacy in medical imaging, this paper proposes a federated learning-based deep learning model using a modified VGG16 architecture. By allowing decentralized model training across multiple clients without sharing raw data, it preserves privacy while still achieving high precision through transfer learning across diverse datasets (Figshare, SARTAJ, Br35H).

10. Applying Deep Learning Techniques in Accurate Brain Tumor Detection

Authors: Chen, Hsuan-Yu, et al. **Published:** 2025 (Engineering Proceedings) **Abstract/Brief:** This study classified brain tumors using CNN, VGGNet19, ResNet101V2, and EfficientNetV2B2 alongside preprocessing and data augmentation. They found that VGGNet19 and customized CNNs excelled in stability and accuracy. The research confirms that deep learning significantly enhances diagnostic efficiency and can assist in reliable clinical decision-making.

Literature Review Summary Table

The following table summarizes the key methodologies, datasets, performance metrics, and pros/cons of the reviewed papers. This provides a quick reference to the current state of research in this domain.

Ref	Methodology Used	Dataset Used	Performance Metrics	Pros	Cons
[1]	Ensemble Attention Mechanism using MobileNetV3 & EfficientNetB7	Figshare, BraTS 2019	Accuracy, ROC, AUC	Very high accuracy, attention improves feature focus	Complex architecture, higher computation
[2]	Federated Learning with Transfer Learning using VGG16	Figshare, SARTAJ, Br35H	Accuracy, Precision, Recall, F1-score	Preserves data privacy, high accuracy	Federated setup is complex
[3]	DenseNet121-based Transfer Learning	Kaggle MRI Dataset	Accuracy, Precision, Recall, F1-score	Efficient feature reuse, high classification accuracy	Needs careful fine-tuning
[4]	Lightweight ResNet models (ResNet-18, 34, 50)	Public MRI Dataset (7023 images)	Accuracy, AUC, Sensitivity, Specificity	Computationally efficient, suitable for real-time use	Slight misclassification in similar tumor types
[5]	EfficientNetB0 Feature Extraction + SVM (EFF_D_SVM)	Public MRI Dataset	Accuracy, Precision, Recall, F1-score	Hybrid DL + ML improves robustness	Two-stage model increases complexity
[6]	Comparison of Pre-trained Models (VGG16, VGG19, ResNet50, InceptionV3)	Brain MRI Dataset	Accuracy, Precision, Recall, F1-score	No manual feature extraction needed	Overfitting risk on small datasets
[7]	Multiple Pre-trained CNN Models with Transfer Learning	Brain Tumor MRI Dataset	Accuracy, Weighted F1-score	Strong generalization, handles class imbalance	Interpretability issues
[8]	Review of CNN-based brain tumor classification methods	BRATS, Figshare, Kaggle (multiple studies)	Accuracy, Sensitivity, Specificity	Good overview of existing DL methods	No own experimental results
[9]	Machine Learning & Transfer Learning based	MRI Brain Tumor Dataset	Accuracy, Precision, Recall, F1-score	High accuracy using modified GoogLeNet;	Limited to binary classification; high computational cost

	Binary Classification			improves robustness	
[10]	LightGBM with modified GoogLeNet and CNN models	Brain Tumor MRI Dataset	Accuracy, Precision, Recall, F1-score	High accuracy with reduced training time	Binary classification; limited interpretability